

Enumitem パッケージ

@monaqa

目次

1. パッケージの概要	?
1.1. 用語集	?
2. enumitem をはじめよう	?
2.1. enumitem パッケージのインストール	?
2.2. パッケージの読み込み	?
2.3. 基本的なコマンドの使用	?
3. Gallery	?
3.1. ラベルの変更	?
3.2. 複雑な箇条書き	?
3.3. 応用 : To Do リスト	?
3.4. 応用 : 定義リスト	?
3.5. ユースケース : 大問と小問からなるレポート	?
4. enumitem の設計思想	?
4.1. 糖衣構文ベースの記法	?
4.2. ブロックベースの記法	?
4.3. ブロックベースの記法の利点	?
4.4. 2 つの記法の併用	?

4.5. enumitem パッケージにおける箇条書きの等価性	?
5. 機能一覧	?
5.1. Enumitem モジュール	?
5.2. itemfmt 型	?

1. パッケージの概要

enumitem は SAT_YSF_I にて豊富な箇条書きや番号付きのリストを提供するパッケージです。標準にもすでに itemize という名前のパッケージが用意されているものの、本パッケージではより自由度の高い箇条書きを提供します。

具体的には、以下のような箇条書きを書くことができます：

- デフォルトで豊富なスタイルのラベルを選択できる
- 番号付き箇条書き環境をネストさせることができる
- ネストごとに箇条書きのスタイルを変更できる
- 箇条書きのラベルの体裁を項目ごとに変更できる
- 定義リストを作成できる
- ユーザ自身がスタイルを拡張できる

本ドキュメントは SAT_YSF_I v0.0.6 及び enumitem パッケージ v3.0.1 の組み合わせでの仕様を述べたものです。旧バージョン、すなわちメジャーバージョンが 0, 1, 2 のいずれかであるものとの互換性は無く、

たとえコマンド名が同じであっても引数のルールが異なることなどがあります。

バージョンの違いに注意してください。

1.1. 用語集

ここでは以下のような用語を用います。

箇条書き いくつかの単語や文章、段落などを分けて書き並べたリストのこと。

本パッケージではそのうち、本文と独立した段落で組まれるものを扱う。

項目 箇条書きを構成する 1 つ 1 つの要素。1 つのラベルと本文で構成され、ネストされた箇条書きの場合は 1 つ以上の子項目を有する場合がある。

番号付き箇条書き 番号や順序のわかる記号を用いてラベルがふられた箇条書き。

番号無し箇条書き 順序のない記号などを用いてラベルがふられた箇条書き。

ラベル 箇条書きの開始を表すために先頭につける記号や文字の

列。 通常、番号付き箇条書きであれば “1.” や “(a)”, 番号無し箇条書きであれば “•”
といったテキストや記号などが用いられる。

2. enumitem をはじめよう

2.1. enumitem パッケージのインストール

enumitem パッケージは Satyrophos によってインストールすることができます。

```
opam install satysfi-enumitem
satyrophos install
```

opam list satysfi-enumitem などのコマンドで バージョン 3 系がインストールされていることを確認したら、次に進みましょう。

2.2. パッケージの読み込み

SATYSFI の文書ファイルやヘッダファイルで外部のパッケージを用いるには以下のようにします。

```
@require: enumitem/enumitem
```

さらに、EnumitemAlias というモジュールも open しておきましょう。これは enumitem パッケージで定義されているモジュールの一つであり、
箇条書きの体裁を変更するための関数、定義リスト専用のコマンドなど便利な機能がまとまっています。
まとめると、文書ファイルの場合は以下のような文言をファイルに書けばよいということになります。「本文」のところは、各クラスファイルの書き方に従ってください。

```
@require: enumitem/enumitem
```

```
% プリアンブル (トップレベル宣言)
```

```
open EnumitemAlias
```

```
in % プリアンブルの終わり
```

```
document (| ... |) '< % この書き方はクラスファイルに応じて変わる
```

```
% 本文
...
>
```

準備ができたなら、本文のどこかに以下のように書いてみましょう。

```
+listing{
  * foo
  * bar
    ** barfoo
    ** barbar
  * baz
}
```

もし正常にパッケージが読み込まれていれば、以下のように出力されるはずです。

- foo
- bar
 - barfoo
 - barbar
- baz

2.3. 基本的なコマンドの使用

enumitem は +listing 及び +enumerate コマンドを提供します。標準の itemize パッケージを使ったことがある人にとっては馴染み深いでしょう。

使い方は標準のコマンドと（オプション引数を除き）ほぼ変わりません。

```
+listing{
  * hoge
  * fuga
    ** fuga1
      *** fuga11
      *** fuga12
    ** fuga2
}
```

- hoge

- fuga
 - fuga1
 - fuga11
 - fuga12
 - fuga2

番号付きの箇条書きのネストにも対応しています。

```
+enumerate{
  * hoge
  * fuga
    ** fuga1
      *** fuga11
      *** fuga12
    ** fuga2
}
```

1. hoge
2. fuga
 - (a) fuga1
 - (i) fuga11
 - (ii) fuga12
 - (b) fuga2

3. Gallery

3.1. ラベルの変更

+listing

コマンドを用いて番号無し箇条書きが書けることは既に見たとおりですが、オプション引数を指定することでラベルを自由に変更することができま

す。以下の例はラベルに `bullet` 及び `white-bullet` を指定した箇条書きであり、ラベルがそれぞれ黒丸、白丸に置き換わっていることがわかります。

```
+listing?:(bullet){
  * 水馬、赤いな。ア、イ、ウ、エ、オ。
  ** 浮藻に小蝦もおよいでる。
}
+listing?:(white-bullet){
  * 水馬、赤いな。ア、イ、ウ、エ、オ。
  ** 浮藻に小蝦もおよいでる。
}
```

- 水馬、赤いな。ア、イ、ウ、エ、オ。
 - 浮藻に小蝦もおよいでる。
- 水馬、赤いな。ア、イ、ウ、エ、オ。
 - 浮藻に小蝦もおよいでる。

`+enumerate` コマンドも同様にオプション引数を用いてラベルを変更できます。

```
+enumerate?:(paren-alpha){
  * 水馬、赤いな。ア、イ、ウ、エ、オ。
  ** 浮藻に小蝦もおよいでる。
  * 柿の木、栗の木。カ、キ、ク、ケ、コ。
  ** 啄木鳥こつこつ、枯れけやき。
  * 大角豆に醋をかけ、サ、シ、ス、セ、ソ。
  ** その魚浅瀬で刺しました。
}
```

- (a) 水馬、赤いな。ア、イ、ウ、エ、オ。
 - (a) 浮藻に小蝦もおよいでる。
- (b) 柿の木、栗の木。カ、キ、ク、ケ、コ。
 - (a) 啄木鳥こつこつ、枯れけやき。
- (c) 大角豆に醋をかけ、サ、シ、ス、セ、ソ。
 - (a) その魚浅瀬で刺しました。

なお、`+listing` と `+enumerate` の違いはオプション引数のデフォルト値のみであるため、

上の例で `+listing?:(paren-alph){ ... }` と書いても同じ結果となります。

もう少し自由度高くラベルを設定したい場合もあるでしょう。

`text-label` 関数を用いると、任意のインラインテキストをラベルとして取り扱うことができます。

```
+listing?:(text-label {一. }){
  * 水馬、赤いな。ア、イ、ウ、エ、オ。
    ** 浮藻に小蝦もおよいでる。
  * 柿の木、栗の木。カ、キ、ク、ケ、コ。
    ** 啄木鳥こつこつ、枯れけやき。
  * 大角豆に醋をかけ、サ、シ、ス、セ、ソ。
    ** その魚浅瀬で刺しました。
}
```

```
一. 水馬、赤いな。ア、イ、ウ、エ、オ。
  一. 浮藻に小蝦もおよいでる。
一. 柿の木、栗の木。カ、キ、ク、ケ、コ。
  一. 啄木鳥こつこつ、枯れけやき。
一. 大角豆に醋をかけ、サ、シ、ス、セ、ソ。
  一. その魚浅瀬で刺しました。
```

`index-label` 関数を用いると、インデックスの値（整数値）を引数に取ってインラインテキストを返すような関数を与えることで対応するラベルを生成することができます。自由度の高い番号付き箇条書きを生成するしたい場合に便利です。

```
+enumerate?:(
  let square-idx idx =
    let it = (idx * idx) |> arabic |> embed-string in
    {\ #it;.\ }
  in
  index-label square-idx
){
  * 水馬、赤いな。ア、イ、ウ、エ、オ。
    ** 浮藻に小蝦もおよいでる。
  * 柿の木、栗の木。カ、キ、ク、ケ、コ。
    ** 啄木鳥こつこつ、枯れけやき。
```

* 大角豆に醋をかけ、サ、シ、ス、セ、ソ。

** その魚浅瀬で刺しました。

}

1. 水馬、赤いな。ア、イ、ウ、エ、オ。

1. 浮藻に小蝦もおよいでる。

4. 柿の木、栗の木。カ、キ、ク、ケ、コ。

1. 啄木鳥こつこつ、枯れけやき。

9. 大角豆に醋をかけ、サ、シ、ス、セ、ソ。

1. その魚浅瀬で刺しました。

ここまで見てきた例はいずれもルールが同じネストレベルのインデックスにのみ依存しており、ラベルが箇条書きのネストの深さに応じて変化する、といったことがありませんでした。しかし、親の項目には番号付きのラベルを、子の項目には番号無しのラベルを振る、といったケースも珍しくありません。

ネストの深さに応じてラベルを変化させたい場合は

`with-depth` 関数を用います。 `with-depth` 関数はラベルのリストを引数に取り、項目のネストの深さが n である項目はリストの n 番目のラベルを用いて組まれます。

```
+enumerate?: (with-depth [paren-alph; text-label {続き : }]) {
```

* 水馬、赤いな。ア、イ、ウ、エ、オ。

** 浮藻に小蝦もおよいでる。

* 柿の木、栗の木。カ、キ、ク、ケ、コ。

** 啄木鳥こつこつ、枯れけやき。

* 大角豆に醋をかけ、サ、シ、ス、セ、ソ。

** その魚浅瀬で刺しました。

}

(a) 水馬、赤いな。ア、イ、ウ、エ、オ。

続き：浮藻に小蝦もおよいでる。

(b) 柿の木、栗の木。カ、キ、ク、ケ、コ。

続き：啄木鳥こつこつ、枯れけやき。

(c) 大角豆に醋をかけ、サ、シ、ス、セ、ソ。

続き：その魚浅瀬で刺しました。

```
+listing?:({| 一. | 二. | 三. |} |> List.map text-label |> with-depth){
  * 五十音
  ** 水馬、赤いな。ア、イ、ウ、エ、オ。
  *** 浮藻に小蝦もおよいでる。
  ** 柿の木、栗の木。カ、キ、ク、ケ、コ。
  *** 啄木鳥こつこつ、枯れけやき。
  ** 大角豆に醋をかけ、サ、シ、ス、セ、ソ。
  *** その魚浅瀬で刺しました。
}
```

一. 五十音

二. 水馬、赤いな。ア、イ、ウ、エ、オ。

三. 浮藻に小蝦もおよいでる。

二. 柿の木、栗の木。カ、キ、ク、ケ、コ。

三. 啄木鳥こつこつ、枯れけやき。

二. 大角豆に醋をかけ、サ、シ、ス、セ、ソ。

三. その魚浅瀬で刺しました。

3.2. 複雑な箇条書き

ここまで挙げた使い方だけでも多くの箇条書きを組むことができますが、より複雑な箇条書きを組みたい場合もあるかもしれません。たとえば

- 1つの箇条書きの途中でラベルのルールを変えたい

と、項目の中に複数の段落や別行立てで数行などを入れたい。

これらは、`+itemize` 及び `+item` というコマンドを用いて組むことができます。

```
+itemize(dot-arabic)<
+item{`+item` では、1 番目の必須引数に書いたものが項目となる。}<
+item{ネストさせることも可能。}<>
+item{2 番目の必須引数に書いたものが子項目となる。}<>
>
+item{
  デフォルトでは `+itemize` の必須引数で指定したラベルが用いられる。
  したがって、ここはアラビア数字で番号付けされている。
}<>
+item?:(paren-roman){ここはローマ数字での番号付け。}<
```

```

+item{`+item` の 1 つ目の必須引数の前でラベルを指定すれば、自身のラベルが変わる。}<>
+item{子の項目はローマ数字にならず、アラビア数字のまま。}<>
>
+item?: (paren-alpha) ({ここはアルファベットでの番号付け。})?: (bracket-arabic)<
+item{
  `+item` の 2 つ目の必須引数の前でラベルを指定すれば、
  子の項目のラベルが全て変わる。
}<>
+item{今回の場合、ネストの中身は角括弧で囲まれたアラビア数字となる。}<
+item{孫項目のラベルも角括弧で囲まれたアラビア数字。}<>
>
>
+item{ここはアラビア数字。}<>
>

```

1. `+item` では、1 番目の必須引数に書いたものが項目となる。

1. ネストさせることも可能。

2. 2 番目の必須引数に書いたものが子項目となる。

2. デフォルトでは `+itemize` の必須引数で指定したラベルが用いられる。したがって、ここはアラビア数字で番号付けされている。

(iii) ここはローマ数字での番号付け。

1. `+item` の 1 つ目の必須引数の前でラベルを指定すれば、自身のラベルが変わる。

2. 子の項目はローマ数字にならず、アラビア数字のまま。

(d) ここはアルファベットでの番号付け。

[1] `+item` の 2 つ目の必須引数の前でラベルを指定すれば、子の項目のラベルが全て変わる。

[2] 今回の場合、ネストの中身は角括弧で囲まれたアラビア数字となる。

[1] 孫項目のラベルも角括弧で囲まれたアラビア数字。

5. ここはアラビア数字。

項目の中に複数の段落を入れたいときにも `+itemize` と `+item` の組み合わせは有用です。

```

+item{
  一つの項目に複数の段落を入れる例。
  `+item` の 1 番目の必須引数に入力されたものが段落となる。
}

```

```

}<
+p{
  しかし、2 番目の必須引数に `+p` コマンドなど通常のコマンドを入れると、
  段落をはじめとしたブロックを後続させることができる。
}
+p{
  段落は当然複数置くことができる。
}
>

+item{
  別行立て数式を配置する例。
}<
+math(${y = a});

+p{
  インラインテキスト内で別行立て数式を配置するコマンドも存在するが、
  マークアップの意味合いは若干異なり、こちらは改段落が行われる。
}
>
>

```

- 一つの項目に複数の段落を入れる例。 `+item` の 1 番目の必須引数に入力されたものが段落となる。

しかし、2 番目の必須引数に `+p` コマンドなど通常のコマンドを入れると、段落をはじめとしたブロックを後続させることができる。

段落は当然複数置くことができる。

- 別行立て数式を配置する例。

$$y = a$$

インラインテキスト内で別行立て数式を配置するコマンドも存在するが、マークアップの意味合いは若干異なり、こちらは改段落が行われる。

`+itemize` 及び `+item` は `+listing` などのコマンドと比較してより冗長な記述を必要とするものの、`+listing` などのコマンドでは表現できない複雑な箇条書きもパワフルに表現できます。詳細は第 4 章も参照。

なお、場合によっては `+listing` と `\sublist` などのコマンドを組み合わせることで複雑な箇条書き

を実現できるケースもあります。

```
+listing{
  * 番号付き箇条書きと番号なし箇条書きの混用。
  ** 以下、番号無し箇条書き。
    \sublist(bullet){
      * ここは番号なし箇条書き。
      * ここは番号なし箇条書き。
      * ここは番号なし箇条書き。
    }
  ** 以下、番号付き箇条書き。
    \sublist(dot-arabic){
      * ここは番号付き箇条書き。
      * ここは番号付き箇条書き。
      * ここは番号付き箇条書き。
    }
}
```

- 番号付き箇条書きと番号なし箇条書きの混用。
 - 以下、番号無し箇条書き。
 - ここは番号なし箇条書き。
 - ここは番号なし箇条書き。
 - ここは番号なし箇条書き。
 - 以下、番号付き箇条書き。
 1. ここは番号付き箇条書き。
 2. ここは番号付き箇条書き。
 3. ここは番号付き箇条書き。

3.3. 応用 : To Do リスト

`enumitem` を用いて To Do リストを実現することもできます。こちらは少し前準備が必要となります。まずはプリアンブル（トップレベル）にて以下のコマンドを定義します。

```
let oalign ib1 ib2 =
  let (wid, ht, dp) = get-natural-metrics ib1 in
  inline-graphics wid ht dp (fun pt -> [draw-text pt ib1; draw-text pt ib2])
let ib-box ctx =
```

```

let ctx = ctx |> set-dominant-narrow-script Kana in
read-inline ctx (string-unexplode [0x2713] |> embed-string)
let ib-checkmark ctx =
  let ctx = ctx |> set-dominant-narrow-script Kana in
  read-inline ctx (string-unexplode [0x2501] |> embed-string)

let-inline ctx \mark-todo = ib-box ctx
let-inline ctx \mark-done = oalign (ib-box ctx) (ib-checkmark ctx)

```

\mark-todo; は未完了タスクを表すラベル「」を、 \mark-done; は完了タスクを表すラベル「」を表しています。

上のコード例では用意されているフォントのグリフを重ね打ちするなどして作成したものの、 画像ファイルを読み込む、 SATySF_I のグラフィックス機能を用いて描画する、といった手法でラベルを用意することも可能です。

これらのコマンドと +itemize や +item コマンドを駆使することで、 以下のように To Do リストを組むことができます。

```

+itemize(text-label {\mark-todo;\ })<
  +item?:(text-label {\mark-todo;\ }){\ミルクを買う。}<>
  +item?:(text-label {\mark-todo;\ }){\The \SATySFIbook を読む。}<>
  +item?:(text-label {\mark-done;\ }){\SATySFI を完全に理解する。}<>
  +item?:(text-label {\mark-todo;\ }){\課題を解く。}<>
>

```

 ミルクを買う。

 The SATySF_Ibook を読む。

 SATySF_I を完全に理解する。

 課題を解く。

とはいえこれはマークアップと呼ぶには少し複雑すぎますね。

もう少し楽に書くため、以下のようなコマンドを追加で定義してみましょう。

ついでに、完了した項目は灰色となるような工夫を入れてみます。

```

let-block +todo-list bt = '< +itemize(text-label {\mark-todo;\ })(bt); >
let-block +todo it = '< +item?:(text-label {\mark-todo;\ })(it)<> >
let-block ctx +done it =

```

```

let ctx = ctx |> set-text-color (Color.gray 0.5) in
read-block ctx '< +item?:(text-label {\mark-done;\ })(it)<> >

+todo-list<
+todo{ミルクを買う。}
+todo{The \SATySF{book} を読む。}
+done{\SATySF{ を完全に理解する。}
+todo{課題を解く。}
>

```

- ミルクを買う。
- The SATySF{book} を読む。
- SATySF{ を完全に理解する。
- 課題を解く。

3.4. 応用：定義リスト

定義リストとはいくつかの用語を並べて説明したり、定義を述べたりする際に用いられる箇条書きです。ラベルには通常その項目が説明する用語名が入り、本文でその説明が書かれる構造となっています。定義リストは箇条書きとしては少し複雑ですが、幸い、今まで見たとおり `enumitem` パッケージにはそういった複雑な箇条書きを表現するだけの十分な機能が備わっています。

最も単純な定義リストは以下のようなものです。

```

+description<
+ditem{用語 1}{ここに用語 1 の説明が入る。}<>
+ditem{用語 2}{
  ここに用語 2 の説明が入る。用語 2 の説明は少し長い。
  用語 2 の説明は少し長い。用語 2 の説明は少し長い。用語 2 の説明は少し長い。
}<>
+ditem{ちょっと長めの用語 3}{
  ここに用語 3 の説明が入る。用語 3 の説明は少し長い。
  用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。
}<
+pn{
  複数行にまたがる説明を入れることもできる。
}
>

```

>

用語 1 ここに用語 1 の説明が入る。

用語 2 ここに用語 2 の説明が入る。用語 2 の説明は少し長い。 用語 2 の説明は少し長い。用語 2 の説明は少し長い。用語 2 の説明は少し長い。

ちょっと長めの用語 3 ここに用語 3 の説明が入る。用語 3 の説明は少し長い。 用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。

複数の段落にまたがる説明を入れることもできる。

新たに登場した `+ditem` コマンドは `+item` と似ているものの、追加で 1 番目にインラインテキストの必須引数を取り、

そこに書かれたテキストをラベルとして表示します。

また、`+ditem` のオプション引数を用いると、ラベル部分の体裁を変更することができます。

以下はラベルを強調して視認性を高めた例です。

```
+description<
+item?:(fun it -> {\emph{#it;}}){用語 1}{ここに用語 1 の説明が入る。}<>
+item?:(fun it -> {\emph{#it;}}){ちょっと長めの用語 3}{
ここに用語 3 の説明が入る。用語 3 の説明は少し長い。
用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。
}<>
>
```

用語 1 ここに用語 1 の説明が入る。

ちょっと長めの用語 3 ここに用語 3 の説明が入る。用語 3 の説明は少し長い。 用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。

実際に用いる場合は、以下のようにプリアンブルでコマンドを定義しておくのが楽です。もし複数段落にまたがる説明を書く予定がないときは、以下のようなコマンドを定義しておけば十分でしょう。

```
let-block +dd dt it-dd = '< +ditem?:(fun it -> {\emph{#it;}}){#dt;}{#it-dd;}<> >
```

上のように定義された `+dd` コマンドを用いれば、`+ditem` をそのまま用いるよりも楽に定義リストを記述できます。

```
+description<
```

```

+dd{用語 1}{ここに用語 1 の説明が入る。}
+dd{用語 2}{
  ここに用語 2 の説明が入る。用語 2 の説明は少し長い。
  用語 2 の説明は少し長い。用語 2 の説明は少し長い。用語 2 の説明は少し長い。
}
+dd{ちょっと長めの用語 3}{
  ここに用語 3 の説明が入る。用語 3 の説明は少し長い。
  用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。
}
>

```

用語 1 ここに用語 1 の説明が入る。

用語 2 ここに用語 2 の説明が入る。用語 2 の説明は少し長い。 用語 2 の説明は少し長い。用語 2 の説明は少し長い。用語 2 の説明は少し長い。

ちょっと長めの用語 3 ここに用語 3 の説明が入る。用語 3 の説明は少し長い。 用語 3 の説明は少し長い。用語 3 の説明は少し長い。用語 3 の説明は少し長い。

3.5. ユースケース：大問と小問からなるレポート

ここで、少々現実的な箇条書きを組んでみましょう。

以下はとある大学生のレポート課題を切り取ったものです。

```

+enumerate?:(with-depth [question; paren-arabic; bullet]){
  * Lebesgue 測度の劣加法性より（中略）、 $\{m\paren{A} = m\paren{B}\}$  が示された。
  * **  $\{B_N = \bigcup_{n=1}^{\infty} A_n\}$  とおく。（中略）
  ** 上限の定義より（中略）
  * **  $\{C_0 = 1\}$  は明らかである。  $\{C_n\}$  は（中略）
  **  $\{C_n\}$  は  $\{C_{n-1}\}$  から適当な部分集合を取り除いてできる集合であるから
      $\{C_n \subset C_{n-1}\}$  となり、 $\{C_n\}$  は減少列である。（中略）
  ** \subitem<
    +item{内測度  $\{m_{J*}\paren{C}\}$ <
      +pn{
         $\{C_n\}$  を構成する 1 つ 1 つの閉区間の長さは（中略）
      }
      +pn{
        したがって  $\{m_{J*}\paren{C} = 0\}$ 。
      }
    }
  >

```

```

+item{外測度  $m_{J^*}(\text{paren}\{C\})$ <
+pn{
+   $\{C_n\}$  を構成する 1 つ 1 つの閉区間の長さは (中略)
+}
+pn{
+  したがって  $m_{J^*}(\text{paren}\{C\}) = 1$ 。
+}
+
+pn{以上より、 $\{C\}$  は Jordan 非可測集合である。}
+
+}

```

設問 1. Lebesgue 測度の劣加法性より (中略)、 $m(A) = m(B)$ が示された。

設問 2. (1) $B_N = \bigcup_{n=1}^{\infty} A_n$ とおく。 (中略)

(2) 上限の定義より (中略)

設問 3. (1) $C_0 = 1$ は明らかである。 C_n は (中略)

(2) C_n は C_{n-1} から適当な部分集合を取り除いてできる集合であるから $C_n \subset C_{n-1}$ となり、 $\{C_n\}$ は減少列である。 (中略)

(3) • 内測度 $m_{J^*}(C)$

C_n を構成する 1 つ 1 つの閉区間の長さは (中略)

したがって $m_{J^*}(C) = 0$ 。

• 外測度 $m_{J^*}(C)$

C_n を構成する 1 つ 1 つの閉区間の長さは (中略)

したがって $m_{J^*}(C) = 1$ 。

以上より、 C は Jordan 非可測集合である。

ここで、question は以下のようにユーザ定義された関数です。

```
let question =
```

```

let labelf idx =
  let num = idx |> arabic |> embed-string in
  {\textbf{設問 #num;}\ }
in
index-label labelf
  |> EnumitemFormat.set-label-inherit
  |> EnumitemFormat.set-indent (fun ctx wid-label -> get-font-size ctx *' 1.0)

```

4. enumitem の設計思想

enumitem v3.0.0 は以下の思想を元に設計されています。

- SAT_YSF_I の文法に備わっている箇条書き用の平易な構文を可能な限り活用する。
- 箇条書き用の構文でカバーできない機能については、専用のコマンドで対応する。
- 関数を用いてラベルの体裁を指定することで、ユーザによる拡張ができるようにする。

簡潔に書ける箇条書きと自由度の高い箇条書きを両立させるため、enumitem パッケージでは箇条書きの木構造を表現する 2 種類の表現方法が用意されています。

4.1. 糖衣構文ベースの記法

1 つは SAT_YSF_I の箇条書きを表す専用の構文をそのまま用いてネストを表現する方法。こちらは標準の

itemize パッケージでも採用されているインターフェースであり、簡単に記述できるのが利点です。ただし、設定できる項目の自由度はあまり高くありません。便宜上、ここではこちらを糖衣構文ベースの記法と呼びます。

```

+listing?:(listing-default-label){
  * aaa
  * bbb
    ** bbb の子 1
    ** bbb の子 2
      *** bbb の子 2 の子 1
  * ccc
    ** ccc の子 1
    ** ccc の子 2
}

```

なお、`+listing` のオプション引数のデフォルトは `listing-default-label` であるため、上のコードの `?:(listing-default-label)` に相当する箇所は省略することができます（次の例との比較のためあえてつけています）。

4.2. ブロックベースの記法

もう 1 つは `SATySFI` の箇条書き専用構文を用いることなく、`+itemize` 及び `+item` のみを用いてブロックテキストベースで箇条書きのネストを表現する方法。星印 `*` だけでアイテムの始まりを示すことができた糖衣構文ベースの記法とは異なり、項目のたびに `+item` を書く必要があります。記法としては冗長になるものの、箇条書きを記述する際の自由度が高いというメリットがあります。こちらをブロックベースの記法と呼びます。

```
+itemize(listing-default-label)<
+item{aaa}<>
+item{bbb}<
+item{bbb の子 1}<>
+item{bbb の子 2}<
+item{bbb の子 2 の子 1}<>
>
>
+item{ccc}<
+item{ccc の子 1}<>
+item{ccc の子 2}<>
>
>
```

ここで示した 2 つのコードは、どちらも最終的には同様の結果となることに注意してください。

4.3. ブロックベースの記法の利点

糖衣構文ベースでもブロックベースでも記述できる先程示したような箇条書きは、より簡潔に書ける糖衣構文ベースの記法のほうが書きやすいといえます。ブロックベースの記法の利点は、糖衣構文ベースの記法と比べて自由度が高く、より複雑なルールに則った箇条書きを柔軟に表現できる点です。

ブロックベースの記法によって自然に実現できるものは、大きく分けて 3 種類あります。1 つ目は、箇条書きの途中で項目の体裁を変更すること。 `+item` コマン

ドの 1 番目のオプション引数によって、自身の要素の体裁のみを変更できます。

```
+itemize(dot-arabic-rec)<
  +item{aaa}<>
  +item?:(paren-alph){bbb}<
    +item{bbb の子 1}<>
    +item{bbb の子 2}<
      +item{bbb の子 2 の子 1}<>
    >
  >
  +item?:(white-bullet){ccc}<
    +item{ccc の子 1}<>
    +item{ccc の子 2}<>
  >
>
```

1. aaa

(b) bbb

2.1. bbb の子 1

2.2. bbb の子 2

2.2.1. bbb の子 2 の子 1

◦ ccc

3.1. ccc の子 1

3.2. ccc の子 2

そして、2 番目のオプション引数によって自身の子要素の体裁全てを変更できます。

```
+itemize(dot-arabic-rec)<
  +item{aaa}<>
  +item({bbb})?:(paren-alph)<
    +item{bbb の子 1}<>
    +item{bbb の子 2}<
      +item{bbb の子 2 の子 1}<>
    >
  >
  +item({ccc})?:(white-bullet)<
    +item{ccc の子 1}<>
```

```
+item{ccc の子 2}<>
>
>
```

1. aaa

2. bbb

(a) bbb の子 1

(b) bbb の子 2

(a) bbb の子 2 の子 1

3. ccc

◦ ccc の子 1

◦ ccc の子 2

2 つ目は、項目の下に任意のブロックテキストを配置できること。

```
+itemize(listing-default-label)<
+item{ここは最初の段落です。ラベルは最初の段落の冒頭に付きます。}<
+pn{
  ここは 2 番目の段落です。
  2 番目以降であれば自由にブロックテキストを挿入できます。
  以下はコードブロックを挿入する例。
}
+code(`aaa`);
+item{このように、途中で箇条書きを挟むことができます。}<
+item{当然ネストさせることもできます。}<>
>
+pn{
  箇条書きを挟んだ後、元の階層に戻って再び段落を再開することができます。
}
>
>
```

- ここは最初の段落です。ラベルは最初の段落の冒頭に付きます。

ここは 2 番目の段落です。 2 番目以降であれば自由にブロックテキストを挿入できます。 以下はコードブロックを挿入する例。

```
aaa
```

- このように、途中で箇条書きを挟むことができます。
- 当然ネストさせることもできます。

箇条書きを挟んだ後、元の階層に戻って再び段落を再開することができます。

3 つめは、ユーザが自由に箇条書きの項目単位でのコマンドを定義できること。

% プリアンブルで定義しておく

```
let-block +genin it = '< +item?:(text-label{下人「」}{#it;」}<> >
```

```
let-block +rouba it = '< +item?:(text-label{老婆「」}{#it;」}<> >
```

```
+itemize(nofmt)<
```

```
+genin{何をしていた。云え。云わぬと、これだぞよ。}
```

```
+genin{
```

```
  己は検非違使の庁の役人などではない。
```

```
  今し方この門の下を通りかかった旅の者だ。
```

```
  だからお前に縄をかけて、どうしようと云うような事はない。
```

```
  ただ、今時分この門の上で、何をして居たのだから、それを己に話しさえすればいいのだ。
```

```
}
```

```
+rouba{
```

```
  この髪を抜いてな、この髪を抜いてな、鬘にしようと思うたのじゃ。
```

```
}
```

```
>
```

下人「何をしていた。云え。云わぬと、これだぞよ。」

下人「己は検非違使の庁の役人などではない。

今し方この門の下を通りかかった旅の者だ。 だからお前に縄をかけて、どうしようと云うような事はない。 ただ、今時分この門の上で、何をして居たのだから、それを己に話しさえすればいいのだ。」

老婆「この髪を抜いてな、この髪を抜いてな、鬘にしようと思うたのじゃ。」

4.4. 2つの記法の併用

2つの記法は組み合わせて使用することもできます。 `+item{}<>` のブロックテキスト中で `+sublist` を用いると

```
+itemize(listing-default-label)<
```

```
  +item{aaa}<>
```

```

+item{bbb}<
  +sublist(listing-default-label){
    * bbb の子 1
    * bbb の子 2
    ** bbb の子 2 の子 1
  }
>
+sublist(listing-default-label){
  * ccc
  ** ccc の子 1
  ** ccc の子 2
}
>

```

+listing のインラインテキスト中で \subitem を用いると、先程の例とは逆に糖衣構文ベースの記法内でブロックベースの +item コマンドなどを用いることができます。

```

+listing{
  * aaa
  * bbb
  \subitem<
    +item{bbb の子 1}<>
    +item{bbb の子 2}<
      +item{bbb の子 2 の子 1}<>
    >
  >
  * ccc
  \subitem<
    +item{ccc の子 1}<>
    +item{ccc の子 2}<>
  >
}

```

また、糖衣構文ベースの記法の中で糖衣構文ベースの記法を作成できる \sublist もあります。こちらを用いると、糖衣構文ベースの記法であってもラベルの体裁を変更しつつネストさせることができます。

```

+listing{
  * aaa
  * bbb

```

```

\sublist(listing-default-label){
  * bbb の子 1
  * bbb の子 2
  \sublist(listing-default-label){
    * bbb の子 2 の子 1
  }
}
* ccc
\sublist(listing-default-label){
  * ccc の子 1
  * ccc の子 2
}
}

```

4.5. enumitem パッケージにおける箇条書きの等価性

enumitem パッケージには様々なコマンドが定義されていますが、実はほぼ全てのコマンドは `+item` というコマンドのみを用いて等価に書き表すことができます。

たとえば `+item` コマンドを用いる全ての箇条書きは全て `+itemize` コマンドで囲むことになっていますが、実はこの `+itemize` も `+item` の一種にすぎません。実際、以下の 2 つのコードは等価です：

```

+itemize(bullet)<
  +item{foo bar}<>
>

+item?: (EnumitemFormat.init)({})?: (bullet)<
  +item{foo bar}<>
>

```

また、`+listing` を用いたコードも、実は内部で再帰的に `+item` を用いたインラインテキストへ書き換えるような実装となっています。

```

+listing?: (bullet){
  * foo
  ** foo1
  ** foo2
  * bar
}

```

```
+item?:(EnumitemFormat.init)({})?:(bullet)<
  +item{foo}<
    +item{foo1}<>
    +item{foo2}<>
  >
  +item{bar}<>
>
```

更に、インラインテキストの中に子構造を記述できる `\sublist` などのコマンドも、最終的には `+item` を用いた木構造のコードにほぼ等価となるよう実装されています。

```
+listing?:(bullet){
  * foo
  \sublist(dot-arabic){
    * foo1
    * foo2
  }
  * bar
  \subitem<
    +item{bar1}<>
    +item{bar2}<>
  >
}
```

```
+item?:(EnumitemFormat.init)({})?:(bullet)<
  +item{foo}<
    +item?:(dot-arabic){foo1}<>
    +item?:(dot-arabic){foo2}<>
  >
  +item{bar}<
    +item{bar1}<>
    +item{bar2}<>
  >
>
```

5. 機能一覧

5.1. Enumitem モジュール

Enumitem

モジュールは本パッケージの根幹となるコマンドを定義します。

最も大切なのは `+item` コマンドであり、上で述べたように Enumitem パッケージで提供される主要なコマンドは ほとんど `+item` を用いたブロックボックス列に展開される仕組みになっています。 `+item` は以下の引数を取ります。

```
+item?:( itemfmt-self )({ text-body })?:( itemfmt-child )< children >
```

引数の詳細は以下のとおりです。

itemfmt-self (itemfmt、オプション)

自分自身の項目の体裁。

省略した場合は、自身の親で設定されたフォーマットが用いられる。

あくまで自分自身の項目のみに影響があり、後述の通り、自身の子要素には反映されない。

text-body (inline-text、必須)

自分自身の項目の本文。ラベルは本文の左端に付く。

空のインラインテキストを指定することもでき

る。 その場合はフォーマットの `display-label-with-empty-body` オプションによってラベルが付くかどうか変化する。

itemfmt-child (itemfmt、オプション)

子要素の項目の体裁のデフォルト値。省略した場合、

子要素は自身の親で設定されたフォーマットを引き継ぐ (`itemfmt-self` の値は用いられない)。

children (block-text、必須)

子要素の項目。空のブロックテキストを指定することもでき、その場合は `block-nil` と同等の扱いとなる。

5.2. itemfmt 型

`itemfmt` 型は `enumitem` パッケージで定義されている最も重要な型であり、 箇条書きの体裁の全てを決定づけるものです。

`itemfmt` 型は `context -> int list -> itemconfig` 型のエイリアスであり、 現在のテキスト処理文脈と、対象となる項目のインデックスの列を与えたときに

ラベルの体裁やインデント量といった設定値をまとめたレコードを返すような関数です。
`itemconfig` は以下のフィールドを持つレコードとなっています。

indent (length -> length)

左側のインデント量を与える関数。より具体的には、「label フィールドの値で決定されるラベルのボックスの横幅を与え、インデント量を返す関数」。

項目の本文は（ラベルの付与される先頭の行を除き）全てここで決定される長さだけインデントされる。たとえば恒等写像 (`fun len -> len`) を与えると、箇条書きの本文はラベルの長さだけインデントされる。

indent-right (length)

右側のインデント量。

label (inline-boxes)

ラベルを表すインラインボックス列。

label-with-empty-body (labelwithemptybody)

項目の中身（項目の直後に続くインラインテキスト）が空だったとき、その項目をどのように描くか。 `labelwithemptybody` は以下の 3 つの定数ヴァリエントからなる代数的データ型である。

LabelDisplay テキストが空でも構わずにラベルを描画する。

LabelInherit 自身はラベルを描画せず、自身の空でない子項目があればそこで一緒に描画させてもらう。

LabelIgnore ラベルを描かず、継承もしない。

margin-top (length)

箇条書きの段落の上に追加であける余白。ただし、`SATYSFI` 標準の機能で設けられる段落間空白はこれと独立に挿入される。

margin-bottom (length)

箇条書きの段落の下に追加であける余白。ただし、`SATYSFI` 標準の機能で設けられる段落間空白はこれと独立に挿入される。

context-body (context)

テキスト処理文脈。

ラベル以外の全ての本文が与えられた関数によって変換される。